

Rajalakshmi Engineering College

Name: Swetha R
Email: 240701550@rajalakshmi.edu.in
Roll no: 240701550
Phone: 9444370187
Branch: REC
Department: CSE - Section 10
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 8.5

Section 1 : Coding

1. Problem Statement

Vikram loves listening to music and wants to create a simple playlist manager using Java Collections. The playlist supports the following operations:

"ADD <song>" Adds the song to the end of the playlist. "REMOVE <song>" Removes the first occurrence of the song from the playlist. If the song is not found, do nothing. "SHOW" Displays all songs in the playlist in order. If the playlist is empty, print "EMPTY". "NEXT" Moves to the next song in the playlist and prints its name. If the playlist is empty, print "EMPTY".

The playlist maintains a "current song" position that starts at the first song when it's added. The NEXT command moves to the next song and prints it, wrapping around to the first song after reaching the last song. When removing songs, the current position adjusts accordingly to maintain

proper navigation.

Help Vikram implement this playlist manager.

Input Format

The first line of the input consists of an integer n, the number of operations.

The next n lines, each containing a command:

- "ADD <song>"
- "REMOVE <song>"
- "SHOW"
- "NEXT"

Output Format

For each "SHOW" command, print the songs in order, separated by spaces.

For each "NEXT" command, print the next song in the playlist.

If no song exists, print "EMPTY".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

ADD song1

ADD song2

SHOW

NEXT

REMOVE song2

SHOW

NEXT

Output: song1 song2

song2

song1

song1

Answer

```
// You are using Java
import java.util.*;
class Node{
    String data;
    Node next;
    Node(String data){
        this.data=data;
        this.next=null;
    }
}
class LinkedList{
    Node head;
    Node current;
    void add(String data){
        Node newnode=new Node(data);
        if(head==null){
            head=newnode;
            current=head;
            return;
        }
        Node temp=head;
        while(temp.next!=null){
            temp=temp.next;
        }
        temp.next=newnode;
    }
    void remove(String data){
        if(head==null){
            return;
        }
        if(head.data.equals(data)){
            head=head.next;
            if(current==head){
                current=head;
            }
            return;
        }
        Node temp=head;
        while(temp.next!=null && !temp.next.data.equals(data)){
            temp=temp.next;
        }
        if(temp.next!=null){
```

```

        if(current==temp.next){
            current=temp.next.next;
        }
        temp.next=temp.next.next;

    }
}
boolean isEmpty(){
    return head==null;
}
void display(){
    Node temp= head;
    if(isEmpty()){
        System.out.println("EMPTY");
    }
    while(temp!=null){
        System.out.print(temp.data);
        temp=temp.next;
    }
}
void move(){
    if(isEmpty()){
        System.out.println("EMPTY");
        return;
    }
    if(head==null){
        current=head;
    }
    else{
        current=current.next;
        if(current==null){
            current=head;
        }
    }
    System.out.println(current.data);
}
}

public class Main{
    public static void main(String args[]){
        Scanner in=new Scanner(System.in);
        int n=in.nextInt();
        in.nextLine();
    }
}

```

```
LinkedList list=new LinkedList();
for(int i=0;i<n;i++){
    String s=in.nextLine();
    if(s.startsWith("ADD")){
        list.add(s.substring(4));
    }
    if(s.startsWith("REMOVE")){
        list.remove(s.substring(7));
    }
    if(s.equals("NEXT")){
        list.move();
    }
    if(s.equals("SHOW")){
        list.display();
    }
}

}
```

Status : Partially correct

Marks : 8.5/10